

Service Layer – Concept & Implementation

1. What is the Service Layer?

The **Service Layer** is a design layer that sits **between the Controller (or UI)** and the **Repository Layer**.

 Its main role:

- Handle **business logic**
- Coordinate between multiple repositories
- Enforce rules and workflows of the application

 **Simple Flow:**

Controller → Service Layer → Repository Layer → Database

2. Why Do We Need a Service Layer?

If you only use repositories, your controller becomes messy because:

- Business logic spreads everywhere
- Code duplication happens
- Hard to maintain and test

 **Service Layer solves this by:**

- Centralizing business rules
 - Keeping controllers thin (clean)
 - Improving testability
 - Supporting scalability (easy to extend logic later)
-

3. Responsibilities of Service Layer

The Service Layer is NOT just a pass-through. It should:

1. Business Logic

Example:

- Calculate discounts
- Validate domain rules
- Apply conditions before saving data

2. Coordination

Example:

- Call multiple repositories
- Combine results

3. Validation (Business-level, not UI-level)

Example:

- "User cannot register twice with same email"

♦ 4. Transactions

Example:

- Save order + payment together (all succeed or fail)
-

4. 🛠 Difference: Repository vs Service

Layer	Responsibility
Repository	Data access (CRUD only)
Service	Business logic + coordination

👉 Example:

- Repository: `GetUserById()`
 - Service: `RegisterUser()` , `PlaceOrder()`
-

5. ⚙ Example (ASP.NET Core)

♦ Repository

```
public interface IUserRepository
{
    User GetByEmail(string email);
    void Add(User user);
}
```

♦ Service Interface

```
public interface IUserService
{
    void RegisterUser(User user);
}
```

♦ Service Implementation

```
public class UserService : IUserService
{
    private readonly IUserRepository _userRepo;
```

```
    public UserService(IUserRepository userRepo)
    {
        _userRepo = userRepo;
```

```

    }

    public void RegisterUser(User user)
    {
        var existingUser = _userRepo.GetByEmail(user.Email);

        if (existingUser != null)
            throw new Exception("User already exists");

        // Business logic
        user.CreatedAt = DateTime.Now;

        _userRepo.Add(user);
    }
}

```

◆ Controller

```

public class AccountController : Controller
{
    private readonly IUserService _userService;

```

```

    public AccountController(IUserService userService)
    {
        _userService = userService;
    }

    [HttpPost]
    public IActionResult Register(User user)
    {
        _userService.RegisterUser(user);
        return RedirectToAction("Login");
    }
}

```

6. 🧩 When Service Layer Becomes Important

You REALLY need it when:

- Logic becomes complex
 - Multiple repositories are involved
 - You want clean architecture (DDD style)
 - You care about testing and scalability
-

7. 🚫 Common Mistakes

- ✗ Making Service Layer just call repository directly (no logic)
- ✗ Putting business logic in controllers
- ✗ Mixing database logic inside service (should stay in repository)

8. 🧪 Testing Advantage

With Service Layer:

- You can mock repositories
- Test business logic independently

9. 🏠 Summary

- Service Layer = **brain of the application**
- Repository Layer = **data access only**
- Controllers = **request handling only**

👉 Clean separation:

Controller → Service → Repository → DB